The Math of Logic

STUDENT EDITION · with Professor Logic

One machine builds every system of logic ever invented. You're going to build them yourself — and never take a single one on faith.

PROFESSOR LOGIC

Hello! Sit anywhere. I'm an owl, I teach logic, and I have exactly one promise for you: by the end of this book you will not *believe* anything I say. You'll *check* it. That's not me being difficult — it's the whole point. Belief is what you do when you can't test something. We're going to test everything.

Here's the deal. Logic isn't a list of rules some ancient philosophers handed down from a mountain. It's a set of *tools people built* — real people, with names and bad days and brilliant ideas — to solve real problems. And once you see the machine underneath, the whole thing stops being scary and starts being *yours*.

How this book works (read this, it's short)

Every important claim wears a **label** telling you how much to trust it and why. No claim gets to hide.

THE TRUST LABELS

FORCED We can prove it by checking every case. The computer file that ships with this book runs the check. No cases fail? Forced.

STIPULATED A *choice* someone made. It could've been different — and we'll show you the other option so you know it was a choice.

HISTORY Something that actually happened — a real person, a real year, a real problem they were staring at.

ANALOGY A comparison to help it click. True *inside the comparison*, never claimed beyond it.

PROFESSOR LOGIC · RIGOR CHECK

See that last label? ANALOGY. I'm going to give you a *ton* of analogies in this book — light switches, group chats, video games, gossip. Analogies are how ideas click. But here's my one rule about them: **an analogy is a bridge, not a house.** Walk across it to reach the idea, then don't try to live on it. When I compare a truth value to a traffic light, I'm not saying truth *is* a traffic light. I'm handing you a bridge. Cross it, get the idea, wave goodbye to the traffic light.

PROFESSOR LOGIC

And one enormous idea, up front, that the whole book keeps coming back to: **a logic is a map, and a map is not the territory.** When you draw a map you *choose* what to show and what to ignore. That's not cheating — that's what maps are *for*. But the second a map starts *demanding you believe it* instead of letting you test it against the real world, it's stopped being a tool and turned into a dogma. Put it down when that happens. Even this book. *Especially* this book.

THE ONE MACHINE — YOU'LL SEE IT EVERYWHERE

Everything in this book is the same three-step move, over and over: **a pattern → pushed through a filter → gives an answer.** Pick what things are allowed to be. Pick how you combine them. Read off what's forced. That's it. That's the whole engine. Watch for it.

THE JOURNEY

CHAPTER 1

What Can Something Even Be?

Before you can do anything with a thing, you have to know its options.

PROFESSOR LOGIC

Let's start with a light switch. Not glamorous, I know. But a light switch is the simplest thing in the world that has what we call a *value* — and value is where *all* of this begins.

A light switch can be ON or OFF. That's the whole list. It can't be "kind of on." So before you do any logic with anything, you ask the first question: **what's it allowed to be?** That list of allowed options is so important it gets a name.

THE CHALKBOARD — Carrier set (a.k.a. "value space") STIPULATED

```
The complete list of values something is allowed to be.
Curly braces { } just mean "here's a collection."

  Light switch:   { ON, OFF }        or  { 1, 0 }
  Traffic light:  { RED, YELLOW, GREEN } or { 0, 0.5, 1 }
  Coin flip:      { HEADS, TAILS }   or  { 0, 1 }
  Your XP level:  { 0, 1, 2, 3, ... }    (infinitely many)
  Temperature:    every number on the line (infinite, smooth)
```

TL;DR

A *carrier set* is just: **the list of everything a thing is allowed to be.** Light switch? Two options. Traffic light? Three. That list is step one of literally everything.

PROFESSOR LOGIC

Why do we swap the words for numbers — ON becomes 1, OFF becomes 0? Because *numbers calculate*. You can't do arithmetic with the word "on." I put YELLOW at 0.5 because it sits in the middle: not fully stop, not fully go, so — halfway between 0 and 1.

But *catch what just happened.* I **chose** to put yellow at exactly 0.5. Someone else might say "nah, yellow really means *get ready to stop*, put it at 0.8." Neither of us is wrong about traffic — we just drew *different maps*. And here's the sentence to tattoo on your brain: **everything that follows is forced once you pick the map. But nothing forces the map.** The choice is yours. The consequences are not.

A bit of history: the number trick is older than you think HISTORY

Turning YES/NO into 1/0 feels obvious now, but someone had to think of it first. **1847:** an almost entirely self-taught English mathematician named **George Boole** — a shoemaker's son who couldn't afford university and taught himself Greek, Latin and math from borrowed books — published a little book arguing that *logic could be done with algebra.* People thought reasoning belonged to philosophers. Boole said: no, it's a *calculation*, and I can prove it with symbols. He was right. Every computer you've ever touched runs on his idea. We'll meet him properly next chapter.

Which values "count"? — designated values

PROFESSOR LOGIC

One more idea and it matters a LOT later. In most carriers, some values *count as a yes* — true, holding, go — and some don't. In { 0, 1 }, the value 1 counts. Obvious. But watch it get interesting with three values.

Take a traffic light, { 0, 0.5, 1 }. Which values "count"? **Depends on the question.** If the question is "should I floor it?" — only GREEN counts, just 1. If the question is "should I be paying attention?" — well, YELLOW *and* GREEN both count now: 0.5 and 1. Same three values. Different question. Different answer about which ones count.

THE CHALKBOARD — Designated values FORCED verified: test_1_1

```
A "designated" value is one that counts as YES for your purpose.
Which ones count is a CHOICE. What follows from that choice is not.

  Carrier { 0, 0.5, 1 }, rule "counts if v > 0":
    counts:         { 0.5, 1 }     (exactly two — checked by listing them)
    doesn't count:  { 0 }
```

TL;DR

Some values count as "yes," some don't. **You pick which** — based on the question you're asking. "Should I go" and "should I pay attention" draw the line in different places on the same traffic light.

TRY IT YOURSELF — CHAPTER 1

Everything here only needs what we just did: writing carrier sets and picking which values count.

1. Write the carrier set (in curly braces) for: (a) a phone on Do-Not-Disturb / Ring / Vibrate; (b) a yes/no poll; (c) a video-game health bar.
2. Your friend says: "Every carrier can be written with just 0 and 1." Give one example that *proves them right* and one that *proves them wrong*.
3. Design a carrier for a homework status: *not started*, *in progress*, *done*. Write the set, the number encoding, and one question where only *done* counts, and one where *in progress* counts too.

ANSWERS

1. (a) { Vibrate, Ring, DND } → a three-option set like { 0, 0.5, 1 } (the numbers are a choice!). (b) { 0, 1 }. (c) a whole range from empty to full, like every number from 0 to 1 — a health bar is smooth, not just three notches (unless your game uses hearts — then it's a small set!).

2. *Right:* any two-option thing (coin, switch) relabels to { 0, 1 }. *Wrong:* a smooth range like a health bar has infinitely many values — you can't squeeze infinity into two.

3. V = { 0, 0.5, 1 }: not started = 0, in progress = 0.5, done = 1. "Can I submit it?" → only *done* (just 1) counts. "Have I actually touched this yet?" → *in progress* and *done* both count (0.5 and 1). Same set, different line.

CHAPTER 2

Three Tools for Everything

NOT, AND, OR — and the surprise that they're a separate choice from your carrier.

PROFESSOR LOGIC

A carrier set just sits there like a box of LEGO. To actually *reason*, you need *operations* — rules that take values in and hand values out. Three of them show up in basically every logic ever invented: **NOT**, **AND**, **OR**.

NOT flips things. AND is strict — it wants *both*. OR is generous — it's happy with *either*. You already use these every single day; you just don't write them on a board.

TL;DR

NOT = the opposite. **AND** = "I need *both* or I'm out" (the strict friend). **OR** = "*either* works, I'm easy" (the chill friend). That's genuinely it.

PROFESSOR LOGIC

Here's the plot twist that trips up almost everybody. You'd think that once you pick your carrier, the formulas for NOT/AND/OR come *attached*, like they're built in. **They don't.** The operations are a *second, totally separate choice*. And I'm not going to argue that at you — I'm going to *calculate* it, because in this book we don't argue, we check.

THE CHALKBOARD — NOT, AND, OR on { 0, 1 } FORCED verified: test_2_1

```
The classic wiring everyone learns first:
  NOT(v) = 1 − v      AND(a,b) = min(a,b)      OR(a,b) = max(a,b)

  NOT: NOT(0)=1,  NOT(1)=0          (flips — obviously)
  AND: min(1,1)=1  min(1,0)=0  min(0,0)=0   (needs both)
  OR:  max(0,0)=0  max(1,0)=1  max(1,1)=1   (either will do)
```

PROFESSOR LOGIC

Now watch me prove the operations are a real choice. I'll keep the *exact same carrier* { 0, 1 } and just... wire AND differently. Instead of min, I'll use *multiplication*.

THE CHALKBOARD — Same carrier, different AND FORCED **verified: test_2_2**

```
On { 0, 1 }:  min(a,b) and (a × b) give the SAME answers every time.
  min(1,1)=1, 1×1=1 ✓   min(1,0)=0, 1×0=0 ✓   min(0,0)=0, 0×0=0 ✓

So on { 0, 1 } you literally cannot tell them apart. BUT —
on { 0, 0.5, 1 }:  min(0.5, 0.5) = 0.5   but   0.5 × 0.5 = 0.25
  Different answers! And 0.25 isn't even in the carrier.
```

PROFESSOR LOGIC · RIGOR CHECK

Feel how sneaky that was? On two values, min and times are *twins* — you'd never know they're different operations. Add one middle value and the mask comes off: they disagree, and multiplication spits out 0.25, which isn't even *allowed* in { 0, 0.5, 1 }. That's a machine that *leaks*. We'll hunt that exact bug in Chapter 3. The lesson for now: **your carrier and your operations are two separate choices, and you don't find out you chose badly until you test.**

The engineer: George Boole and the algebra of thought HISTORY

1815–1864. George Boole grew up poor in Lincoln, England, teaching himself the mathematics that snooty universities wouldn't teach a shoemaker's kid. In *1854* he published *The Laws of Thought*, and the wild claim inside it was this: the rules of reasoning — AND, OR, NOT — behave like *algebra*. You can calculate with them. For decades it was treated as a neat curiosity. Then, in *1937*, a grad student named **Claude Shannon** realized Boole's two-value algebra was the perfect language for electrical switches: on/off, 1/0. Every microchip on Earth is now built from Boole's AND, OR, NOT. A self-taught kid's "curiosity" became the nervous system of the modern world. Boole never saw a computer. He built its grammar anyway.

TRY IT YOURSELF — CHAPTER 2

Only uses carriers (Ch 1) and the three operations with min/max/(1−v).

1. On { 0, 1 }, compute by hand: NOT(0), AND(1,0), OR(1,0), OR(0,0).
2. The "strict friend / chill friend" test: for a party, AND = "I'll come only if *both* my friends come," OR = "I'll come if *either* comes." If friend A comes (1) but friend B doesn't (0), do you show up under AND? Under OR?
3. On { 0, 0.5, 1 } with AND = min: compute AND(0.5, 1) and AND(0.5, 0). Are the answers still inside the carrier? (This is the "does it leak?" check.)

ANSWERS

1. NOT(0)=1; AND(1,0)=min(1,0)=0; OR(1,0)=max(1,0)=1; OR(0,0)=0.

2. AND: no (you needed *both*, only A came). OR: yes (either was enough). Strict friend stays home; chill friend parties.

3. AND(0.5,1)=min=0.5 ✓ in carrier. AND(0.5,0)=min=0 ✓ in carrier. min never leaks — every answer it gives was already one of the inputs. (That's exactly why min is the good choice and multiplication wasn't.)

CHAPTER 3

Laws That Can't Break

Some truths are forced. Others are just printed on the map you happen to be holding. Also: why you audit before you admire.

PROFESSOR LOGIC

Big question. "A thing can't be both true and false at once." Did somebody *decide* that rule? Or does it fall out of the arithmetic all by itself? The answer — the second one, it falls out — is one of the most freeing ideas in this whole book. Because a *decided* rule could've gone another way. A rule that *falls out of the arithmetic* literally cannot be different, as long as your choices stand.

But — new fine print, and it's the most important fine print in the book — **"forced" always means "forced *given your carrier and your operations.*"** Two choices, and then no more freedom. Change the choices, and different things get forced. Keep that leash on the word "forced" and you'll never get fooled.

TL;DR

Some rules you *chose* (could be otherwise). Some rules the math *forces on you* (can't be otherwise, once you've chosen your carrier + operations). Telling those two apart is basically the superpower this whole book gives you.

First — the audit. Always audit before you admire.

PROFESSOR LOGIC

Before I show you any laws, I'm going to show you a *broken machine* — because the nicest-looking machine in this whole subject is secretly broken, and almost nobody catches it just by reading. Remember multiplication from last chapter? "AND = multiply" reads perfectly on paper. Let's actually *run the check* on { 0, 0.5, 1 }.

THE CHALKBOARD — The pretty machine fails the audit FORCED verified: test_3_0

```
Carrier { 0, 0.5, 1 },  NOT = 1−v,  AND = multiply,  OR = max

  Check every AND: ... AND(0.5, 0.5) = 0.5 × 0.5 = 0.25
  0.25 is NOT in { 0, 0.5, 1 }. The machine LEAKS.

  Reads perfectly. Dead on arrival. Your eyes won't catch it.
  The CODE catches it. That's what "check everything" buys you.
```

THE CHALKBOARD — The repair (this machine has a name) FORCED verified: test_3_1

```
Keep { 0, 0.5, 1 } and NOT = 1−v. Just swap AND back to min.
  Check every case → no leaks. Sealed. Closed.

  This repaired machine is famous. It's called Kleene's K3,
  and a real person built it for a real reason (Chapter 6).
```

PROFESSOR LOGIC · RIGOR CHECK

This is the heartbeat of the whole book, so let me say it plainly: **a thing can look completely correct and be completely broken.** "AND = multiply" is *gorgeous* on paper. It's also garbage — it leaks values that don't exist in your carrier. You didn't catch it by admiring it. You caught it by *auditing* it. Admire second. Audit first. Every single time.

Now — the laws themselves

THE CHALKBOARD — Law of Non-Contradiction (LNC) FORCED verified: test_3_2

```
LNC says: AND(v, NOT v) should hit the BOTTOM (a "no") for every v.
"Nothing is both true and its own opposite at the same time."

  On { 0, 1 }:
    v=0:  min(0, NOT 0)=min(0,1)=0 ✓     v=1: min(1,0)=0 ✓
    Every value checks out → LNC is FORCED in { 0, 1 }.

  On { 0, 0.5, 1 }:
    v=0.5:  NOT(0.5)=0.5,  min(0.5,0.5)=0.5  (that's not the bottom!)
    One failure → LNC is NOT forced once you add a middle value.
```

THE CHALKBOARD — Law of Excluded Middle (LEM) FORCED verified: test_3_3

```
LEM says: OR(v, NOT v) should hit the TOP (a "yes") for every v.
"Everything is either true or false — no third option."

  On { 0, 1 }:
    v=0: max(0,1)=1 ✓   v=1: max(1,0)=1 ✓  →  FORCED.
  On { 0, 0.5, 1 }:
    v=0.5: max(0.5,0.5)=0.5 (not the top) → NOT forced.
```

PROFESSOR LOGIC

Look what just happened, because it's *wild*. Those two laws — "nothing's both true and false," "everything's either true or false" — feel like the bedrock of reality itself, right? Handed down from the universe. **Nope.** They're forced in the two-value world, and they *quietly stop being forced* the instant you allow a middle value. They were never cosmic laws. They were consequences of a *choice* to only have two values. Change the carrier, change what's forced.

THE IDEA TO WALK AWAY WITH

"Forced" and "chosen" are different, and telling them apart is everything. **Excluded middle isn't a law of the universe — it's a law of the { 0, 1 } map.** Pick a different map (add a middle value) and it's gone. Nobody broke reality. You just changed carriers.

TRY IT YOURSELF — CHAPTER 3

Only uses: carriers, min/max/(1−v), and checking a law by testing every value.

1. The "audit" move: On { 0, 0.5, 1 } with AND = min, check whether AND ever leaks a value outside the carrier. Test AND(0,0.5), AND(0.5,0.5), AND(0.5,1). Any leaks?
2. Check LNC on { 0, 1 } yourself, both values, showing the min each time.
3. Your friend insists "everything is either true or false, that's just how truth works." Using the traffic-light carrier { 0, 0.5, 1 }, show them one value where LEM fails. What does that prove — that your friend is wrong about reality, or wrong about *which map* they're standing on?

ANSWERS

1. min(0,0.5)=0 ✓, min(0.5,0.5)=0.5 ✓, min(0.5,1)=0.5 ✓ — all still in the carrier. No leaks. min is safe because its answer is always just the smaller input, which was already allowed.

2. v=0: min(0, NOT 0)=min(0,1)=0 ✓. v=1: min(1, NOT 1)=min(1,0)=0 ✓. Both hit bottom → LNC forced on { 0,1 }.

3. v=0.5: OR(0.5, NOT 0.5)=max(0.5,0.5)=0.5, not the top. LEM fails. It proves your friend is wrong about *which map they're standing on* — LEM is true on the two-value map, false on the three-value one. Reality didn't change; the carrier did.

CHAPTER 4

The Three Knobs

The control panel hiding behind every logic ever built. Learn the knobs, build any logic you want.

PROFESSOR LOGIC

Here's where it all snaps together. Every logic system in this book — and honestly, most of them ever invented — is built from turning exactly **three knobs**. That's the control panel. Once you can see the knobs, logic stops being a pile of things to memorize and becomes a machine you *operate.*

THE CONTROL PANEL

Knob 1 — V: what values are allowed? (the carrier, Chapter 1)

Knob 2 — G: how do you combine them? (the operations, Chapter 2)

Knob 3 — θ (theta): where's the line between "counts" and "doesn't"? (designated values, Chapter 1)

That's it. **V, G, θ.** Three knobs. Turn them different ways, get different logics. Every single one.

TL;DR

Every logic = three settings. **What can things be** (V), **how you mix them** (G), and **where the "yes" line sits** (θ). Learn to turn three knobs, build infinite logics. That's the trade of a logic engineer.

PROFESSOR LOGIC

Think of it like building a character in a game. V is your available stats. G is your move-set — how the stats combine into actions. θ is the difficulty threshold — where "pass" becomes "fail." Same three sliders, wildly different characters. A logic is a *build.*

THE CHALKBOARD — Two logics, same knobs, different settings FORCED
verified: test_4_1

```
CLASSICAL LOGIC — the one you grew up with:
   V = { 0, 1 }      G = (1−v, min, max)      θ = "counts if v = 1"
   → LEM forced, LNC forced. The tidy two-value world.

KLEENE'S K3 — one knob nudged:
   V = { 0, 0.5, 1 }   G = (1−v, min, max)      θ = "counts if v = 1"
   → LEM no longer forced. Just added one value to Knob 1.

Same G, same θ. One change to V. A different logic falls out.
```

PROFESSOR LOGIC · RIGOR CHECK

Don't let anyone tell you "classical logic is *the* logic and the others are weird knockoffs." That's reification — mistaking one setting of the knobs for the whole machine. Classical logic is a *great* build. It's also just... one build. { 0, 1 } with min/max and a strict threshold. Knowing it's a setting, not a law of heaven, is exactly what lets you build better tools for jobs it's bad at. The map is not the territory. The build is not the machine.

TRY IT YOURSELF — CHAPTER 4

Only uses V, G, θ and everything from Chapters 1–3.

1. Name the three knobs from memory, and say in your own words what each one does.
2. I give you a logic: V = { 0, 0.5, 1 }, G = (1−v, min, max), θ = "counts if v = 0.5 or v = 1." Which knob did I change compared to Kleene's K3 above? (Look carefully — it's not V or G.)
3. You want a logic for a fire alarm where BOTH "alarm" and "warning beep" should count as "take action." Using V = { 0, 0.5, 1 } (silent / beep / alarm), where do you set the θ knob?

ANSWERS

1. V = what values are allowed. G = how you combine them (NOT/AND/OR). θ = where the line is between "counts as yes" and "doesn't."

2. The θ knob. V and G are identical to K3 — only the threshold moved, from "just 1 counts" to "0.5 and 1 count." (Hold onto this: moving *only* θ turns out to build a whole different famous logic. Chapter 7.)

3. Set θ to "counts if v = 0.5 or v = 1" — so both the beep and the full alarm trigger action. Same carrier, you just drew the yes-line lower.

CHAPTER 5

Everything Costs Something

Every idea you add to a system charges a price somewhere else. There is no free upgrade.

PROFESSOR LOGIC

Here's a law that isn't about logic symbols at all — it's about *engineering*, and it's the most grown-up idea in the book: **you never add something for free.** Every value you add, every power you gain, gets paid for somewhere else in the system. Logicians learn this the hard way, over and over.

You already *saw* the bill get charged and didn't notice. Adding the middle value 0.5 to your carrier gave you something great — the ability to say "maybe," "undecided," "not yet." But check the receipt: the moment you added it, **excluded middle stopped being forced.** You bought "maybe." You paid with a law you used to have for free. That's the trade. There's always a trade.

TL;DR

No free lunch. Every new power costs an old certainty. Want a "maybe" value? Great — you just gave up "everything's either true or false." The question is never "is this upgrade free?" (it isn't). It's "is the trade worth it *for my job*?"

PROFESSOR LOGIC

Think about your phone. Every app you add costs battery. Every feature costs complexity. Every shortcut you take costs some flexibility later. You already live by this law everywhere *except* where people pretend logic is a magic realm of free truths. It isn't. It's engineering. Engineering has a budget.

THE CHALKBOARD — The receipt for the middle value FORCED verified: test_5_1

```
You added 0.5 to your carrier. Here's the itemized bill:

  GAINED:  a way to say "maybe / undecided / not yet"   (+)
  PAID:    Law of Excluded Middle is no longer forced   (−)
           (v=0.5: OR(0.5, NOT 0.5) = 0.5, not the top)

  Not a bug. Not a mistake. A PRICE. You chose to pay it
  because "maybe" was worth more to you than that law.
```

PROFESSOR LOGIC · RIGOR CHECK

Whenever someone sells you a system that's "all upside, no downside, strictly better than everything," check your wallet — they're either hiding the bill or they haven't audited their own machine. Real tools have real trade-offs, and honest engineers *name* them. When this book adds a value, it shows you the receipt. Demand that from every system you're ever handed, including this one.

TRY IT YOURSELF — CHAPTER 5

Only uses everything through Chapter 4 — carriers, operations, laws, knobs, and now "trade-offs."

1. In your own words: what did you *gain* and what did you *pay* when you added 0.5 to the carrier?
2. Give a real-life "no free lunch" trade from your own life (phone, sports, games, school) where getting one thing cost you another.
3. Someone offers you a logic that "keeps every classical law AND adds a maybe value, no downsides." Based on this chapter, what's your first move — believe them, or audit them? What would you check first?

ANSWERS

1. Gained: the power to represent "maybe / undecided." Paid: excluded middle stopped being forced (at v=0.5, "true or false" no longer lands on a clean "yes").

2. Lots of good ones — e.g. a heavier bat hits harder (gain) but swings slower (cost); more screen brightness (gain) drains battery (cost); a hard class boosts your GPA weight (gain) but eats your time (cost).

3. Audit them, obviously. First check: run their AND on the middle value and see if a law quietly broke or the machine leaks — because *something* was paid, and an honest engineer can tell you exactly what. If they can't name the price, they haven't checked their own machine. (Sneak peek: there IS a clever way to keep both classical laws with three values — Łukasiewicz found it — but it charges a *different* bill. Next chapter.)

CHAPTER 6

A Third Option: Maybe

Two engineers, the same middle value, wired two different ways — for two very different reasons.

PROFESSOR LOGIC

We've been playing with 0.5 for a few chapters. Now let's meet the two people who took it *seriously* — and here's the beautiful part: they built the *same* three values into *different* machines, because they were solving *different problems.* Same LEGO, different builds. This is the clearest look you'll get at "the operations are a choice."

Engineer #1: Jan Łukasiewicz, and a war on "the future is already decided"

HISTORY

1917–1920, Poland. Jan Łukasiewicz (say it "woo-kah-SHEV-itch") was bothered by an ancient, creepy argument: *if it's already true today that you'll do something tomorrow, then isn't tomorrow already locked? Where's your free will?* Classical two-value logic seemed to force the future to be already true or already false — a done deal. Łukasiewicz hated that. In a famous *1918 farewell lecture* he declared what amounted to a rebellion against that locked-in universe. His weapon? **A third truth value** for things not yet decided — "possible," "open," 0.5. He built the first serious three-valued logic to make *room for a future that isn't written yet.*

PROFESSOR LOGIC

Now here's the engineering. Łukasiewicz *wanted* to keep the classical laws even with three values. Remember from Chapter 5 — that should cost something. So how'd he pull it off? He wired AND and OR a *special* way — the "strong" way — that makes truth *expensive*. Watch.

THE CHALKBOARD — Łukasiewicz's clever wiring (Ł3) FORCED **verified: test_6_1**

```
Instead of AND = min, he used a "strong" AND that adds and clamps:
   AND(a,b) = the amount by which a+b beats 1  (0 if it doesn't beat it)

  Two "maybes" together:  AND(0.5, 0.5): does 0.5+0.5 beat 1? No, it ties.
     → AND(0.5, 0.5) = 0   (two half-truths make... nothing!)

  And with this wiring, LEM and LNC BOTH survive at 0.5. He kept his laws.
```

TL;DR

Łukasiewicz wanted three values *and* the old laws. He got them by making AND stingy: two "maybes" combine to a flat "no." Weird? Yes. But it's the price he chose to pay — and it kept the laws he cared about. **Different bill, not no bill.**

Engineer #2: Stephen Kleene, and the computation that hasn't finished

HISTORY

1938–1952, USA. Stephen Kleene (say it "KLAY-nee") wasn't worried about free will. He was one of the founders of the theory of *computation* — what computers can and can't figure out. His middle value meant something totally different: **"undefined — this computation hasn't returned an answer yet."** Think of a loading spinner that might never stop. Not true, not false — *pending.* (Fun fact: Kleene also invented the pattern-matching syntax — the `*` "Kleene star" — that's inside basically every search bar and programming language today. He did it on a break from proving deep things about infinity, partly from his family farm in Maine, which he considered his real home.)

PROFESSOR LOGIC

Kleene wired his middle value as a *taint that spreads.* If part of your calculation is still "loading" (0.5), then combining it with stuff keeps the loading going. He used plain *min* and *max* — the same operations from Chapter 2 — and just let 0.5 propagate. That's the machine we called "K3" back in Chapter 3 when we repaired the leaky one. It was Kleene's all along.

THE CHALKBOARD — Same middle, two machines FORCED **verified: test_6_2**

```
Both use V = { 0, 0.5, 1 }. The ONLY difference is the AND wiring:

  ŁUKASIEWICZ (Ł3):  AND(0.5, 0.5) = 0    → keeps LEM & LNC
  KLEENE (K3):       AND(0.5, 0.5) = 0.5  → LEM not forced

One cell of one table. That's the entire difference between two
famous logics. THAT is what "the operations are a choice" means.
```

PROFESSOR LOGIC · RIGOR CHECK

Please don't file these under "which one is *right*." That's the wrong question — it's like asking whether a hammer is more correct than a screwdriver. Łukasiewicz's machine is right for reasoning about an *open future.* Kleene's is right for reasoning about *computations that might not finish.* Different jobs, different tools, both audited, both honest about their bill. A logic engineer doesn't ask "which logic is true." They ask "which logic fits *this* job." Reifying one as The Truth is exactly the mistake this whole book is training you to catch.

TRY IT YOURSELF — CHAPTER 6

Only uses: the three values, min/max, the "strong AND" defined right above, and reading a law by testing values.

1. Kleene's K3 uses AND = min. Compute AND(0.5, 0.5) the Kleene way and the Łukasiewicz way ("does 0.5+0.5 beat 1?"). Confirm you get 0.5 and 0.
2. In one sentence each: what problem was Łukasiewicz solving, and what problem was Kleene solving?
3. You're designing a logic for a food-delivery app's order status: *delivered* (1), *on the way* (0.5), *failed* (0). Is "on the way" more like Łukasiewicz's "undecided future" or Kleene's "still computing"? (Either answer can be right — defend it.)

ANSWERS

1. Kleene: min(0.5,0.5)=0.5. Łukasiewicz: 0.5+0.5=1, which does *not* beat 1, so the strong AND gives 0. Two different machines, two different answers, same inputs.

2. Łukasiewicz: making room for a future that isn't already true or false (free will vs. a locked-in universe). Kleene: representing a computation that hasn't returned an answer yet ("undefined / still loading").

3. Great case for Kleene: "on the way" is like "still computing — answer pending, will resolve to delivered or failed." Also defensible as Łukasiewicz: "the outcome genuinely isn't decided yet." The point isn't the "right" answer — it's that *you now choose your logic to fit the job.*

CHAPTER 7

When Systems Break (On Purpose)

Paradoxes aren't magic. They're a carrier being asked for a value it doesn't have — and one engineer who decided that was a feature.

PROFESSOR LOGIC

Time for the scariest word in logic: **paradox.** People treat paradoxes like haunted houses — spooky, unsolvable, proof that reality is broken. I'm going to show you they're nothing of the kind. A paradox is just a machine being handed a job its carrier can't do. Once you see the knobs, the ghost turns back into a gear.

The famous one: **"This sentence is false."** Is it true? If it's true, then it's false. If it's false, then it's true. Round and round forever. Spooky!

TL;DR

The Liar paradox ("this sentence is false") isn't magic. It's a sentence that equals its own NOT. In a two-value world, nothing can equal its own opposite, so it spins forever. The "paradox" is just your carrier not having a value that can sit still there.

PROFESSOR LOGIC

Let's actually *find* where it breaks. The Liar says: my value equals NOT my value. So we're hunting for a value `v` where `v = NOT(v)`. Let's check our carriers.

THE CHALKBOARD — Hunting the Liar's home FORCED **verified: test_7_1**

```
The Liar needs a value where  v = NOT(v).  Where can it live?

  On { 0, 1 }:   NOT(0)=1 (nope, 0≠1)   NOT(1)=0 (nope, 1≠0)
     → NO home. The Liar has nowhere to sit. It spins forever.
     THAT spinning is the "paradox."

  On { 0, 0.5, 1 }:  NOT(0.5) = 1 − 0.5 = 0.5   ✓ !!
     → 0.5 = NOT(0.5). The Liar sits down at 0.5 and stops spinning.
```

PROFESSOR LOGIC

Do you SEE that? The paradox was never a crack in reality. It was a **two-value carrier being asked for a value it didn't stock.** Give the system a middle value — one that's its own opposite — and the Liar just... sits there at 0.5, perfectly calm. The haunted house was a storage problem. You needed a third shelf.

PROFESSOR LOGIC · RIGOR CHECK

This is the anti-reification lesson at full volume. People say "the Liar paradox proves logic is broken / reality is contradictory / the universe is mysterious." No. It proves *one carrier* (the two-value one) can't represent *one kind of sentence.* That's a limit of a *map*, not a wound in the *world.* Every time you hear "this proves reality itself is paradoxical," translate it to "this map ran out of shelves" and check whether a different map has the shelf. Usually it does.

The engineer: Graham Priest, who designated the contradiction HISTORY

1979, and still working today. Most logicians treated the Liar as something to *quarantine.* The philosopher **Graham Priest** asked a genuinely rebellious question: *what if we just... let some contradictions be acceptable?* Remember Knob 3, θ — the line for what "counts"? Priest took Kleene's exact K3 machine and did *one thing:* he moved θ so that the middle value **counts as true.** Same V, same G as Kleene — he only turned the θ knob. Suddenly "this sentence is both true and false" is an *allowed, usable* answer instead of a crash. His logic is called **LP**, and it powers real systems that have to keep working even when their data contradicts itself (imagine a database where two sources disagree — you don't want the whole thing to explode).

THE CHALKBOARD — Priest's one move: turn θ FORCED **verified: test_7_2**

```
Start from Kleene's K3.  Change NOTHING about V or G.
  Just move θ:  "counts if v = 1"  →  "counts if v = 0.5 or v = 1"

  Now the Liar's value (0.5) COUNTS. A contradiction becomes a
  usable answer instead of a system crash. This logic is called LP.

Remember Chapter 4, exercise 2? You already found this knob-setting.
You built the front half of Priest's logic without knowing its name.
```

PROFESSOR LOGIC

And of course — Chapter 5, say it with me — **everything costs something.** Priest's move blocks the paradox-crash, but it charges a bill: in LP, one of your most trusted reasoning moves gets shaky (the one where "A is true, and A-leads-to-B, therefore B"). He gained crash-resistance. He paid with a piece of clean deduction. Worth it for a contradictory database. Maybe not for your geometry homework. *Right tool, right job.*

TRY IT YOURSELF — CHAPTER 7

Only uses: the three values, NOT = 1−v, and moving the θ knob (all taught).

1. Show, by computing NOT for each value, that the Liar (needs v = NOT v) has no home in { 0, 1 } but does in { 0, 0.5, 1 }.
2. In your own words: is a paradox more like "reality is broken" or "this map ran out of shelves"? Use the Liar to explain.
3. Priest changed exactly one of the three knobs (V, G, θ) to build LP from Kleene's K3. Which knob? What did the change let the middle value do?

ANSWERS

1. { 0,1 }: NOT(0)=1≠0, NOT(1)=0≠1 — no value equals its own NOT, so the Liar spins (no home). { 0,0.5,1 }: NOT(0.5)=0.5, so 0.5 equals its own NOT — the Liar sits at 0.5.

2. "Ran out of shelves." The Liar only "breaks" the two-value map, which has no self-opposite value. Add that shelf (0.5) and it sits calmly. Reality didn't change — the carrier gained a slot. Calling it "reality is broken" is reifying one map's limit into a cosmic fact.

3. The θ knob (the threshold). Moving it to let 0.5 count as "true" turned the middle value from a crash into a usable answer — contradictions become acceptable instead of exploding the system.

CHAPTER 8

One Machine, Many Masks

The big reveal — and your final challenge.

PROFESSOR LOGIC

Look back at the road. Classical logic. Kleene's K3. Łukasiewicz's Ł3. Priest's LP. Four famous logics, built by four people across a century, in different countries, for totally different reasons — free will, computation, paradox-proofing. And here's the thing I've been walking you toward the whole time:

They're all the same machine. Three knobs, turned to different settings. That's all any of them ever were.

THE CHALKBOARD — The whole book on one board FORCED verified: test_8_1

```
Every logic you met = the same three knobs (V, G, θ):

  CLASSICAL:      V={0,1}       G=(1−v,min,max)      θ: 1 counts
  KLEENE K3:      V={0,0.5,1}   G=(1−v,min,max)      θ: 1 counts
  ŁUKASIEWICZ Ł3: V={0,0.5,1}   G=(1−v,strongAND,..) θ: 1 counts
  PRIEST LP:      V={0,0.5,1}   G=(1−v,min,max)      θ: 0.5 & 1 count

Same machine. Four masks. You can read every difference off the knobs.
```

TL;DR

You didn't learn four logics. You learned *one machine* and four settings of it. Classical, K3, Ł3, LP differ only in V, G, or θ — and you can now point to exactly which knob makes each one what it is. That's not memorization. That's understanding.

PROFESSOR LOGIC · THE LAST RIGOR CHECK

So here's what I actually want you to leave with — bigger than any single logic. **A formal system is a tool you build, not a truth you find.** You pick the values. You wire the operations. You set the threshold. Then — and only then — some things get *forced*, and you can prove exactly which ones by checking every case. The forcing is real and rigorous. The *choices underneath it* were yours.

The one deadly mistake — the one this whole book was secretly about — is **reification:** forgetting you chose, and treating your map like it's the territory. "Excluded middle is just *true*." "Classical logic is *the* logic." "This paradox proves reality is broken." Every one of those forgets that a person, somewhere, turned a knob. Stay awake to the knobs and you can't be fooled — not by a logic, not by a textbook, not by me.

THE WHOLE BOOK, IN ONE BREATH

Pick your values. Wire your operations. Set your threshold. Check what's forced. Never forget you chose. Do that, and you're not memorizing logic anymore — you're *engineering* it. And you can catch anyone, including a friendly owl, trying to slip a dogma past you dressed as a law.

YOUR FINAL CHALLENGE — BUILD AND AUDIT YOUR OWN LOGIC

Uses everything in the book. Nothing new. You have every tool you need.

1. **Pick a job.** Choose something from your life that needs a logic: a group-chat "are we hanging out?" tracker, a game's status effects, a "is my essay done?" checker — anything with values.
2. **Turn the three knobs.** Write your V (the allowed values), your G (how NOT/AND/OR work — you can borrow min/max), and your θ (which values count as "yes").
3. **Audit it.** Check: does your AND ever leak a value outside your carrier? (Test a few cases.) If it leaks, fix it — swap to min, like Kleene did.
4. **Find what's forced.** Test LEM and LNC on your carrier. Are they forced, or did you pay them away for a middle value?
5. **Name your price.** In one sentence: what did your logic gain, and what did it cost? (If you can't name a cost, audit again — there's always one.)
6. **Stay honest.** Write one sentence admitting your logic is a *map* — what does it choose to ignore about the real situation?

THERE'S NO SINGLE RIGHT ANSWER — BUT HERE'S WHAT A STRONG ONE LOOKS LIKE

Job: group-chat hangout tracker. V = { 0, 0.5, 1 } = out / maybe / in. G = NOT is 1−v, AND = min ("are BOTH of us in?"), OR = max ("is EITHER in?"). θ = "counts as a plan if v = 1" (only firm yeses) — or move it to 0.5 for "counts if anyone's a maybe." *Audit:* min never leaks ✓. *Forced?* With the middle value, LEM isn't forced — "you're either in or out" fails for the "maybe" person. *Price:* gained "maybe," paid the clean in-or-out law. *Map honesty:* it ignores *why* someone's a maybe, how strongly, and whether they'll flake — real friends are richer than three values. The logic is a useful map, not the friendship.

PROFESSOR LOGIC

That's it. That's the whole thing. You came in thinking logic was a rulebook handed down from on high, and you're leaving able to *build* logics, *audit* them, *price* them, and *catch* anyone who forgets they're tools. You didn't take a single thing on faith. You checked it all.

Go turn some knobs. And whenever a system — any system — starts *demanding* you believe it instead of letting you test it, you know exactly what to do.

Put it down. Even this one. *Especially* this one.

A note for teachers and the curious. Every chalkboard in this book is stamped with a test name (like `test_7_2`). Those refer to the companion code file that ships with the full Math of Logic project — each stamped claim is checked there by complete enumeration over the carrier. The history is real: George Boole (1815–1864) and *The Laws of Thought* (1854); Claude Shannon's 1937 link to switching circuits; Jan

Łukasiewicz's three-valued logic (1918–1920); Stephen Kleene's strong three-valued logic (1938–1952) and the Kleene star; Graham Priest's *Logic of Paradox* (LP, 1979). Names, dates, and motivations are historical framing; the mathematics is what the tests verify. As Professor Logic would insist: check it yourself.